

Avance Informe Caso semestral

Proyecto de Arquitectura de Microservicios

Tienda Retail Lumina

Asignatura: Desarrollo Fullstack III

Docente: Ignacio Andrés Cuturrufo González

Integrantes: Dylan Carroza, Jorge Ruiz y Benjamín Valencia

Fecha: 20-03-2026

Tienda Retail Lumina.....	1
1. Introducción.....	3
1.1 Propósito.....	3
1.2 Alcance.....	4
1.3 Definiciones y siglas.....	4
2. Descripción General.....	5
2.1 Contexto del negocio.....	5
2.2 Usuarios del sistema.....	6
2.3 Problema a resolver.....	7
3. Requerimientos del Sistema.....	7
4. Diseño de Arquitectura (SECCIÓN PRINCIPAL).....	8
4.1 Selección de patrones de arquitectura.....	9
4.2 Propuesta de microservicios.....	10
4.3 Diagrama de arquitectura.....	11
4.4 Estrategias y herramientas.....	11
5. Consideraciones de Calidad.....	13
5.1 Seguridad.....	13
5.2 Privacidad.....	13
5.3 Sostenibilidad.....	14
6. Evaluación de la Arquitectura.....	14
6.1 Cumplimiento de requerimientos.....	14
6.2 Ventajas.....	15
6.3 Limitaciones.....	15
7. Conclusión.....	15
8. Referencias.....	16

1. Introducción

1.1 Propósito

Propósito Central de la Solución

El propósito principal de rediseñar la plataforma de Lumina utilizando una arquitectura de microservicios es **erradicar la crisis operativa generada por su actual sistema monolítico y rígido**. Al migrar hacia esta nueva infraestructura, Lumina busca optimizar de forma integral sus procesos de logística y venta, recuperando la confianza de los clientes y eliminando las inconsistencias de información que actualmente provocan retrasos críticos.

Objetivos Clave y Beneficios del Enfoque de Microservicios

El diseño basado en microservicios abordará las "dolencias" actuales de la empresa a través de los siguientes enfoques estratégicos:

- **Desacoplamiento y Escalabilidad:** Se transiciona de un sistema monolítico inestable a una arquitectura escalable y desacoplada.
- **Aislamiento de Dominios de Negocio:** El sistema se dividirá en microservicios independientes, incluyendo: Usuarios, Productos, Bodega y Carrito.
- **Estabilidad en el Proceso de Venta:** Al independizar módulos críticos como el "Carrito", se evitará que el carrito pierda los productos seleccionados.
- **Mitigación de Riesgos Legales:** La mejora específica y aislada del módulo de pagos resolverá los errores que han derivado en problemas legales con los consumidores.
- **Optimización Logística en Tiempo Real:** El microservicio de "Bodega" permitirá contar con actualizaciones en tiempo real.

- **Gestión de Inventario:** Este enfoque facilitará la implementación de alertas automáticas para escasez o stock crítico.
 - **Mejora en la Gestión de Datos:** Un microservicio dedicado a "Usuarios" solucionará los fallos en el registro, deteniendo la pérdida constante de datos y clientes potenciales.
 - **Agilidad Frontend-Backend:** La nueva arquitectura utilizará un BFF (Backend For Frontend) para conectar la interfaz responsiva con los microservicios independientes vía JPA.
 - **Renovación Tecnológica:** Esto permitirá modernizar el frontend para que sea responsivo, interactivo y personalizable (como el modo claro/oscuro) sin afectar la lógica de negocio subyacente.
-

1.2 Alcance

Incluye: Diseño y desarrollo de una arquitectura escalable y desacoplada compuesta por microservicios de Usuarios, Productos, Bodega y Carrito. Implementación de un frontend moderno, responsivo y personalizable (modo claro/oscuro) mediante un BFF (Backend For Frontend).

Queda fuera: La migración física de servidores de datos antiguos, campañas de marketing digital y el desarrollo de aplicaciones móviles nativas (se prioriza web responsiva).

1.3 Definiciones y siglas

- **API:** Interfaz de programación de aplicaciones para la comunicación entre servicios.

- **Microservicio:** Enfoque arquitectónico que divide una aplicación en servicios pequeños e independientes.
 - **REST:** Estilo de arquitectura de software utilizado para servicios web.
 - **BFF (Backend For Frontend):** Capa intermedia que conecta la interfaz con los microservicios independientes.
 - **JPA (Java Persistence API):** Estándar para la persistencia de datos entre los servicios y la base de datos.
-

2. Descripción General

2.1 Contexto del negocio

Tipo de empresa

- Tienda online la cual se dedica a las ventas online de ropa con marcas como Adidas, Nike, NorthFace, entre otros. La cual cuenta también con delivery propio de la empresa.

Problema actual

- El sistema cuenta con muchas caídas en horarios “picks” en la página lo cual hace que se sature y no tenga un correcto funcionamiento.
- Al registrar datos de usuarios o actualizar su información dentro del sistema se pierden muchos datos la cual genera mucha incertidumbre y no otorga esa seguridad a los usuarios.
- Un catálogo de productos a falta de información o tiene un desorden en cuanto a sus características respectivas.

- En momento de faltas de productos no cuenta con un sistema que automatice una alarma a bodega de problemas en cuanto al stock de ciertos productos.
- Sistema monolítico que colapsa todo el sistema y al momento de caer una sección de la página caería todo el sistema por completo.

Necesidad principal

- Automatizar las notificaciones a bodega.
- Cambiar el sistema monolítico a un sistema de microservicios para un mejor manejo de la página, generando menos carga y así facilitar los manejos de problemas dentro del sistema.
- Organizar el manejo de información en cuanto a los productos en stock y automatizar la información que se requiera del producto dentro del catálogo de la tienda.
- Mejorar el manejo de información de los usuarios, arreglar la conexión entre la base de datos y la página.
- Arreglar los PUT y POST del sistema para solucionar conflictos de registros de usuarios o al querer realizar una actualización de información dentro del sistema.

2.2 Usuarios del sistema

Administrador

El rango con mayor control, capaz de editar y organizar todo además de tener acceso a información y funciones exclusivas para dirigir la aplicación

Bodeguero

Se encarga de abastecer los productos necesarios para la tienda, actualizar la cantidad de productos disponibles, introducir toda la información relacionada al producto y encargado de realizar las entregas de productos a los delivery para las entregas a realizar.

Cliente

Su función dentro del sistema es registrar su información dentro del sistema al crear una cuenta como **“Nombres, ciudad, fecha de nacimiento, run, dirección y número de celular”**, en caso de no contar con los datos actualizados y requiera cambiar información contará con una sección, podrá realizar una suma de productos a un carrito en la cual contará con una pasarela de pagos para una diversidad de medios de pagos, podrá contar con una sección de solicitudes en caso de presentar algún problema o necesita alguna ayuda y contará con un catálogo de productos disponibles a elegir en gusto del consumidor.

2.3 Problema a resolver

Al transicionar de un sistema monolítico a uno de microservicios se busca el resolver los problemas de rendimiento en horarios saturados de clientes además de estabilizar el programa al separar la carga de uso de los múltiples sistemas, esto no solo mejorará el rendimiento si no que además facilitará la mantención del programa al no tener que interrumpir proceso en uso mientras se trabaja en otro.

3. Requerimientos del Sistema

3.1 Requerimientos funcionales

- El sistema debe permitir el registro y actualización de perfiles de usuario (Nombre, ciudad, RUN, etc.).
 - El sistema debe gestionar un catálogo de productos con información detallada y actualizaciones en tiempo real.
 - El sistema debe automatizar alertas de stock crítico hacia el departamento de bodega.
 - El sistema debe tener un carrito de compras persistente.
-

3.2 Requerimientos no funcionales

Escalabilidad: Capacidad de crecer modularmente según la demanda de tráfico.

Disponibilidad: Minimizar las caídas del sistema mediante el aislamiento de fallos por microservicio.

Rendimiento: Carga rápida de la interfaz y respuesta ágil del backend mediante el BFF.

Seguridad: Implementación de protocolos modernos de autenticación y protección de datos sensibles.

4. Diseño de Arquitectura (SECCIÓN PRINCIPAL)

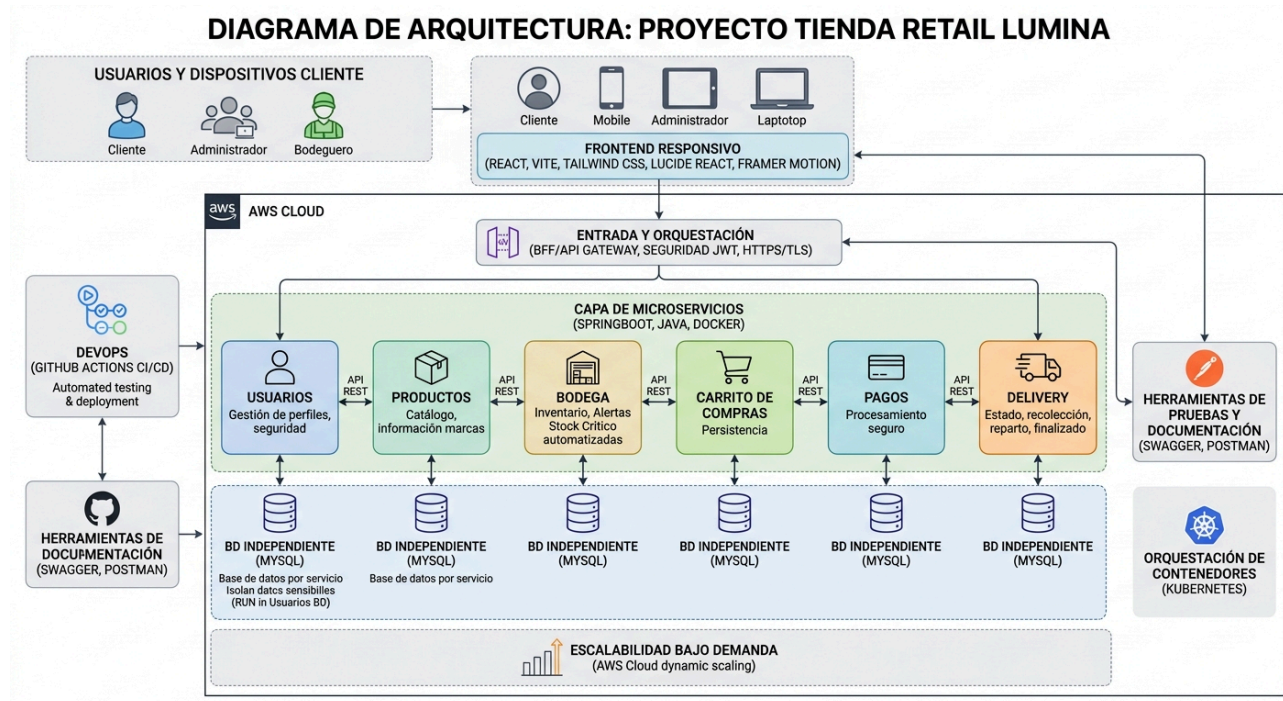
4.1 Selección de patrones de arquitectura

- **Patrón de Microservicios:** Se dividirá el sistema monolítico en dominios de negocio independientes (Usuarios, Productos, Bodega, Carrito, Pagos y Delivery). Esto resolverá directamente el problema de las caídas masivas en horarios "peak". Si un servicio falla, el resto del sistema seguirá operando de manera estable.
- **BFF (Backend For Frontend) / API Gateway:** Se utilizará un BFF para centralizar y orquestar las peticiones entre el frontend y los diferentes microservicios del backend. Esto reduce la complejidad en el lado del cliente y optimiza el rendimiento general de la plataforma.
- **Base de datos por servicio:** Para lograr un verdadero desacoplamiento, cada microservicio tendrá su propia base de datos. Esto evita bloqueos, mejora la escalabilidad y asegura que la saturación de un módulo (como el catálogo) no afecte los registros de usuarios.

4.2 Propuesta de microservicios

Microservicio	Responsabilidad
Usuarios	Gestión de registros, perfiles y seguridad.
Productos	Gestión del catálogo e información de marcas.
Bodega	Control de inventario y alertas de stock.
Carrito de compras	Almacena los productos elegidos de los usuarios.
Pagos	Procesa de manera segura las transacciones monetarias.
Delivery	Se encarga de dar estado del producto como en recolección, en reparto y finalizado.

4.3 Diagrama de arquitectura



4.4 Estrategias y herramientas

- **Frontend (React, Vite, Tailwind CSS, Lucide React, Framer Motion):** Se elige *React* junto a *Vite* para lograr una construcción rápida y un renderizado ágil de la interfaz, vital para la retención de clientes. *Tailwind CSS* permitirá el desarrollo de un diseño altamente responsivo y personalizable (ej. modo claro/oscuro) sin sobrecargar el proyecto. *Framer Motion* ofrece animaciones más interactivas con el usuario mejorando su experiencia, mientras que *Lucide* es una manera más fácil de utilizar iconos dentro de *react*.
- **Backend (Java y SpringBoot):** Se elige *SpringBoot* por ser el estándar más robusto en la industria para el desarrollo rápido y seguro de microservicios RESTful.

- **Base de Datos (MySQL):** Motor relacional elegido por su alta fiabilidad transaccional, garantizando la consistencia mediante *JPA* en procesos de pago y carrito.
- **Comunicación e Integración (REST, GitHub CI/CD):** La comunicación se hará mediante *API REST* para mantener la simpleza y compatibilidad. *GitHub Actions (CI/CD)* automatizará las pruebas y el despliegue continuo de las actualizaciones.
- **Infraestructura (Docker, AWS Cloud):** *Docker* permitirá aislar cada microservicio en contenedores, evitando conflictos de dependencias. *AWS* proporcionará la escalabilidad necesaria para que el sistema crezca bajo demanda durante los horarios saturados.
- **Documentación y Pruebas (Swagger, Postman):** *Swagger* se utilizará para documentar las APIs de manera clara, mientras que *Postman* será vital para solucionar los problemas actuales de los métodos PUT y POST en el registro de usuarios.

¿Por qué éstas herramientas?

Porque son herramientas orientadas a la escalabilidad, la seguridad y el rendimiento. Por parte del front una iteración rápida y una entrega de interfaz ligera, el backend proporciona una base altamente tipada y segura. Esta combinación permite que la parte visual sea flexible ante cambios del mercado, mientras que la lógica y los datos permanecen protegidos por una arquitectura de grado empresarial. También la parte de automatización en cuanto a CI/CD y la documentación de Swagger disminuye errores humanos mediante pruebas, elimina la fricción en la comunicación entre equipos (Front/Back) al tener API claramente definidos y probables, y se acelera la resolución de incidencias (debugging) mediante el uso de estándares industriales como **Postman**, reduciendo drásticamente los tiempos de mantenimiento. Por parte del uso de la Nube se puede escalar componente físicos (Carrito, registros o pagos) de manera

independiente sin necesidad de replicar toda la aplicación, Dockers por su facilidad de implementar los lenguajes a utilizar y con todas sus dependencias para que no haya conflictos en otros sistemas remotos, y finalmente Kubernetes para la orquestación la cual decide cuántos contenedores se requieren funcionando en los momentos de altas demandas.

5. Consideraciones de Calidad

5.1 Seguridad

- **Autenticación y Autorización:** Se implementará un sistema de seguridad (ej. JWT - JSON Web Tokens) para verificar la identidad de los usuarios y garantizar que cada rol (Administrador, Bodeguero, Cliente) acceda únicamente a los módulos e información que le corresponden.
 - **Protección de datos:** Todo el tráfico deberá viajar bajo protocolos HTTPS/TLS, y las contraseñas se almacenarán cifradas para evitar vulnerabilidades.
-

5.2 Privacidad

- **Manejo de datos sensibles y Separación de información:** El patrón de base de datos por servicio garantizará que la información personal de los clientes (RUN, dirección, etc.) esté aislada estrictamente en el microservicio de Usuarios. De esta forma, un empleado de Bodega no tendrá acceso innecesario a los datos de facturación.
-

5.3 Sostenibilidad

- **Escalabilidad y Uso de recursos:** Al desplegarse en AWS con Docker, los recursos se utilizarán de forma dinámica. Solo los microservicios con alta demanda escalarán (por ejemplo, el Carrito), evitando malgastar servidor en procesos que no lo requieren.
 - **Mantenimiento:** El código fragmentado permite a los desarrolladores identificar y corregir errores específicos sin tener que desconectar o reiniciar la tienda completa.
-

6. Evaluación de la Arquitectura

6.1 Cumplimiento de requerimientos

- **Funcionales:** La arquitectura soluciona la gestión de registros delegándola al servicio "Usuarios", asegura un catálogo preciso a través del servicio "Productos", y permite al microservicio "Bodega" automatizar alertas de stock independientemente de la interfaz web.
 - **No funcionales:** Se cumple con la alta disponibilidad mediante el aislamiento de fallos, con la escalabilidad mediante contenedores en la nube, y con el rendimiento ágil a través del patrón BFF.
-

6.2 Ventajas

- Se erradican las caídas totales de la plataforma y se estabiliza la experiencia del cliente.
- Agilidad para actualizar o modificar secciones del frontend sin riesgo de romper la lógica comercial.

- Recuperación de la confianza del consumidor al no existir pérdida de datos de cuentas ni de carritos de compra.
-

6.3 Limitaciones

- El manejo de múltiples bases de datos introduce complejidad a la hora de realizar transacciones distribuidas (requiriendo patrones como Saga).
 - El despliegue y monitoreo inicial requiere de una curva de aprendizaje técnica superior comparada con el sistema monolítico anterior.
-

7. Conclusión

El rediseño de la plataforma de Tienda Retail Lumina hacia una arquitectura orientada a microservicios representa la solución técnica definitiva a la crisis operativa del modelo monolítico. Al separar las responsabilidades en dominios aislados, se ataca directamente el problema de las caídas masivas en horarios de alto tráfico y la pérdida de datos de clientes. Incorporando herramientas modernas como React, SpringBoot y la infraestructura de AWS Cloud, el proyecto no solo garantizará un entorno estable, responsivo y seguro, sino que también sentará una base tecnológica escalable y mantenible. Esto permitirá optimizar la gestión de inventario, agilizar el proceso de ventas y, en última instancia, recuperar la confianza de los consumidores de Lumina.

8. Referencias

- Amazon Web Services. (2024). *AWS Cloud Documentation*. <https://docs.aws.amazon.com/>

- Docker Inc. (2024). *Docker Documentation*. <https://docs.docker.com/>
- GitHub. (2024). *GitHub Actions Documentation*. <https://docs.github.com/en/actions>
- Meta Open Source. (2024). *React: The library for web and native user interfaces*. <https://react.dev/>
- Oracle. (2024). *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/refman/8.0/en/>
- Tailwind Labs. (2024). *Tailwind CSS: Rapidly build modern websites without ever leaving your HTML*. <https://tailwindcss.com/>
- The Kubernetes Authors. (2024). *Kubernetes Documentation*. <https://kubernetes.io/docs/home/>
- Vite. (2024). *Vite: Next Generation Frontend Tooling*. <https://vitejs.dev/>
- VMware. (2024). *Spring Boot Documentation*. Spring. <https://spring.io/projects/spring-boot>